

05_ejecucion-local-y-docker

[#linux](#)[#bash](#)[#python](#)[#fastapi](#)[#docker](#)[#PostgreSQL](#)[#SQLAlchemy](#)[#backend](#)[#frontend](#)[#SIEM](#)

1 Objetivo de la nota

Esta nota explica las dos formas principales de ejecutar el laboratorio SIEM MVP:

1. Ejecución local del backend con Python.
2. Ejecución contenerizada con Docker.

También se incluye la forma de servir el frontend y la relación entre backend, base de datos, variables de entorno y navegador.

El objetivo es entender cómo se pone en marcha el proyecto durante el desarrollo y cómo podría ejecutarlo otra persona a partir de la entrega.

2 Contexto general de ejecución

El proyecto está organizado en varias capas:

```
backend/  
  ↓  
API FastAPI  
  
frontend/  
  ↓  
interfaz HTML, CSS y JavaScript  
  
docker/  
  ↓  
servicios contenerizados  
  
PostgreSQL  
  ↓  
base de datos del laboratorio
```

La ejecución completa necesita que estén disponibles:

- Backend FastAPI.
- Base de datos PostgreSQL.
- Variables de entorno.
- Frontend servido desde un puerto compatible con CORS.

Flujo general:

```
PostgreSQL  
  ↓  
backend FastAPI  
  ↓  
frontend  
  ↓  
usuario
```

3 Ejecución con Docker

La forma más reproducible de ejecutar el laboratorio es mediante Docker Compose.

El proyecto incluye:

```
docker/compose.yml  
backend/Dockerfile
```

```
.env.example
```

La idea es que Docker levante los servicios necesarios sin depender demasiado del entorno local.

Conceptualmente, Docker se encarga de:

- Crear el contenedor del backend.
- Crear el contenedor de PostgreSQL.
- Exponer la API en el puerto configurado.
- Exponer Adminer si está definido en compose.yml.
- Conectar servicios mediante red interna.

4 Archivos implicados en Docker

Los archivos principales son:

```
backend/Dockerfile
docker/compose.yml
.env.example
docker/.env
```

Función de cada uno:

```
backend/Dockerfile
  ↓
define cómo se construye la imagen del backend

docker/compose.yml
  ↓
define servicios, puertos, volúmenes y variables

.env.example
  ↓
plantilla de variables de entorno

docker/.env
  ↓
archivo real de variables para Docker, no entregable si contiene valores locales
```

5 Dockerfile del backend

El Dockerfile del backend contiene:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY . /app

CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

Este archivo construye una imagen Python para ejecutar FastAPI.

Flujo:

```
python:3.12-slim
  ↓
crea imagen base
```

```
WORKDIR /app
↓
define carpeta interna

COPY requirements.txt
↓
copia dependencias

pip install
↓
instala librerías

COPY .
↓
copia código backend

uvicorn app.main:app
↓
arranca FastAPI
```

6 Comando de arranque del backend en Docker

La línea:

```
CMD ["uvicorn", "app.main:app", "--reload", "--host", "0.0.0.0", "--port", "8000"]
```

ejecuta el backend con Uvicorn.

Desglose:

```
uvicorn
↓
servidor ASGI

app.main:app
↓
módulo app/main.py, variable app

--reload
↓
recarga automática en desarrollo

--host 0.0.0.0
↓
escucha conexiones desde fuera del contenedor

--port 8000
↓
puerto interno del backend
```

El uso de `--reload` es propio de desarrollo. Para producción se eliminaría, pero en este laboratorio MVP es razonable.

7 Variables necesarias para Docker

El archivo `.env.example` define:

```
POSTGRES_DB=siem
POSTGRES_USER=siem
POSTGRES_PASSWORD=change_me
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem

API_PORT=8000
```

```
ADMINER_PORT=8080

APP_VERSION=0.1.0
GIT_SHA=unknown
BUILD_TIME=unknown
```

Estas variables permiten que Docker y el backend sepan:

- Qué base de datos crear.
- Qué usuario usar.
- Qué contraseña usar.
- Cómo conectar FastAPI con PostgreSQL.
- En qué puerto publicar la API.
- En qué puerto publicar Adminer.
- Qué información mostrar en /info.

8 Importancia de DATABASE_URL en Docker

La variable más importante para el backend es:

```
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem
```

Dentro de Docker, el host de PostgreSQL es:

```
db
```

Esto se debe a que Docker Compose permite que los servicios se resuelvan por nombre.

Por tanto:

```
api
  ↓
se conecta a

db
  ↓
servicio PostgreSQL
```

No se usa `localhost` porque, dentro del contenedor de la API, `localhost` sería el propio contenedor de la API, no la base de datos.

9 Ejecución con Docker Compose

Desde la raíz del proyecto, el flujo habitual sería:

```
cd ~/siem-lab
```

Después, según la ubicación del archivo Compose:

```
docker compose -f docker/compose.yml up --build
```

Este comando:

- Lee `docker/compose.yml`.
- Construye la imagen del backend si hace falta.
- Levanta los servicios.
- Muestra logs en pantalla.

Si se quiere dejar ejecutando en segundo plano:

```
docker compose -f docker/compose.yml up --build -d
```

Para parar los servicios:

```
docker compose -f docker/compose.yml down
```

10 Comprobación del backend

Una vez levantado el entorno, se puede comprobar el backend con:

```
curl http://localhost:8000/health
```

Respuesta esperada:

```
{
  "status": "ok",
  "db": "ok"
}
```

Este endpoint valida dos cosas:

- La API responde.
- La conexión con PostgreSQL funciona.

También se puede comprobar:

```
curl http://localhost:8000/info
```

Este endpoint devuelve información como:

```
name
version
git_sha
build_time
utc_now
```

11 Swagger

FastAPI genera documentación automática.

Una vez levantado el backend, se puede abrir:

```
http://localhost:8000/docs
```

Desde Swagger se pueden probar endpoints como:

```
GET /health
GET /info
GET /events
POST /ingest
GET /rules
POST /rules
GET /alerts
PATCH /alerts/{alert_id}
GET /metrics
```

Swagger es útil durante el desarrollo porque permite probar la API sin escribir comandos `curl`.

1 2 Adminer

Si Docker Compose levanta Adminer, normalmente estaría disponible en:

```
http://localhost:8080
```

El puerto depende de:

```
ADMINER_PORT=8080
```

Adminer permite consultar la base de datos visualmente.

Sirve para revisar tablas como:

```
events
rules
alerts
alembic_version
```

Es una herramienta útil para validar que las operaciones realizadas desde la API realmente se almacenan en PostgreSQL.

1 3 Ejecución local del backend

Además de Docker, el backend puede ejecutarse localmente con Python.

La estructura ya incluye:

```
backend/venv/
backend/requirements.txt
```

El flujo habitual sería:

```
cd ~/siem-lab/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Después habría que asegurarse de que existe una variable `DATABASE_URL` válida.

Ejemplo:

```
export DATABASE_URL="postgresql+psycopg://siem:change_me@localhost:5432/siem"
```

Y arrancar FastAPI:

```
uvicorn app.main:app --reload
```

1 4 Diferencia entre ejecución local y Docker

La principal diferencia está en el host de la base de datos.

En Docker

```
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem
```

El host es:

```
db
```

porque PostgreSQL está en otro servicio de Docker Compose.

En local

```
DATABASE_URL=postgresql+psycopg://siem:change_me@localhost:5432/siem
```

El host suele ser:

```
localhost
```

porque PostgreSQL estaría instalado o publicado en la máquina local.

1 5 Ventajas de Docker en este proyecto

Docker es la opción más adecuada para una entrega reproducible.

Ventajas:

- No obliga al evaluador a instalar PostgreSQL manualmente.
- Reduce problemas de versiones.
- Usa la misma configuración entre máquinas.
- Levanta varios servicios con un solo comando.
- Aísla el entorno del sistema operativo.

En un proyecto académico como este, Docker ayuda a que la corrección sea más sencilla.

1 6 Ventajas de ejecución local

La ejecución local también tiene ventajas durante el desarrollo:

- Permite depurar más rápido.
- Facilita probar cambios pequeños.
- Permite usar el entorno virtual directamente.
- Puede ser más cómoda para ejecutar tests.

Pero requiere más configuración manual:

- Tener PostgreSQL disponible.
- Tener DATABASE_URL correcta.
- Instalar dependencias.
- Activar venv.

1 7 Servir el frontend

El frontend no necesita compilación.

Se puede servir con Python desde la carpeta `frontend`:

```
cd ~/siem-lab/frontend
python3 -m http.server 5173
```

Después se abre en el navegador:

```
http://localhost:5173/index.html
```

La página de detalle se abre con:

```
http://localhost:5173/alert.html?id=1
```

El puerto `5173` es importante porque el backend tiene CORS configurado para permitir:

```
http://localhost:5173
```

1 8 Relación con CORS

En `backend/app/main.py`, el backend permite el origen:

```
allow_origins=["http://localhost:5173"]
```

Esto significa que el frontend debe servirse desde:

```
http://localhost:5173
```

Si se abre directamente como archivo local:

```
file:///...
```

o desde otro puerto, el navegador podría bloquear las peticiones por CORS.

Por eso es recomendable servirlo con:

```
python3 -m http.server 5173
```

1 9 Flujo completo de ejecución

El flujo completo sería:

1. Levantar backend y base de datos con Docker.
2. Comprobar `/health`.
3. Abrir Swagger en `/docs`.
4. Servir frontend en puerto 5173.
5. Abrir `index.html`.
6. Crear reglas o ingestar eventos desde Swagger/curl.
7. Consultar alertas desde el frontend.
8. Abrir detalle de alerta.
9. Cambiar estado `open/ack/closed`.

Visualmente:

```
docker compose
  ↓
PostgreSQL + FastAPI

python3 -m http.server 5173
  ↓
frontend

navegador
  ↓
consulta API
  ↓
muestra alertas
```

2 0 Comprobación de salud

El primer endpoint que conviene probar es:

```
curl http://localhost:8000/health
```

Si responde:


```
{
  "status": "ok",
  "db": "ok"
}
```

significa que la API y la base de datos están operativas.

Si falla, las causas habituales son:

- Contenedor de PostgreSQL no levantado.
- DATABASE_URL incorrecta.
- Backend no levantado.
- Puerto 8000 ocupado.
- Migraciones no aplicadas.

2.1 Comprobación de información

El endpoint:

```
curl http://localhost:8000/info
```

sirve para comprobar información básica del backend.

Devuelve datos como:

```
name
version
git_sha
build_time
utc_now
```

Este endpoint es útil para validar que la API está respondiendo y que las variables de entorno de versión se están leyendo.

2.2 Crear una regla de prueba

Para validar el flujo, se puede crear una regla:

```
curl -X POST http://localhost:8000/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "High severity auth",
  "enabled": true,
  "source": "auth",
  "severity_min": 5
}'
```

Esto crea una regla que detecta eventos de origen `auth` con severidad igual o superior a 5.

2.3 Enviar un evento de prueba

Después se puede enviar un evento coincidente:

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "auth",
  "severity": 5,
  "message": "High severity authentication event",
  "meta": {
    "host": "server-01"
  }
}'
```

```
}  
'
```

Si la regla coincide, el backend debería crear:

- Un Event.
- Una Alert.

2.4 Consultar alertas

Las alertas se consultan con:

```
curl http://localhost:8000/alerts
```

O desde el frontend:

```
http://localhost:5173/index.html
```

Si existe una alerta, la tabla debería mostrarla.

Desde ahí se puede abrir:

```
alert.html?id=ID_DE_ALERTA
```

2.5 Actualizar estado de una alerta

Desde la API:

```
curl -X PATCH http://localhost:8000/alerts/1 \  
-H "Content-Type: application/json" \  
-d '{  
  "status": "ack"  
'
```

También se puede cerrar:

```
curl -X PATCH http://localhost:8000/alerts/1 \  
-H "Content-Type: application/json" \  
-d '{  
  "status": "closed"  
'
```

O reabrir:

```
curl -X PATCH http://localhost:8000/alerts/1 \  
-H "Content-Type: application/json" \  
-d '{  
  "status": "open"  
'
```

Este flujo valida la parte SOC básica:

```
open → ack → closed
```

2.6 Consultar métricas

El endpoint:

```
curl http://localhost:8000/metrics
```

permite comprobar contadores generales:

```
events_total
rules_total
rules_enabled
alerts_total
alerts_by_status
alerts_by_group_key_top
```

Sirve para validar que eventos, reglas y alertas se están contabilizando correctamente.

2.7 Ejecutar tests

El proyecto contiene tests en:

```
backend/tests/
```

Para ejecutarlos en local:

```
cd ~/siem-lab/backend
source venv/bin/activate
pytest
```

Si se ejecutan dentro de Docker, dependerá de cómo esté definido el servicio en `compose.yml`.

Conceptualmente, pytest valida partes del backend sin necesidad de probar todo manualmente.

2.8 Archivos que afectan a la ejecución

Archivos críticos:

```
backend/requirements.txt
backend/Dockerfile
backend/app/main.py
backend/app/db/database.py
backend/alembic.ini
docker/compose.yml
.env.example
frontend/assets/app.js
```

Función:

```
requirements.txt
↓
instala dependencias

Dockerfile
↓
construye backend

main.py
↓
arranca FastAPI

database.py
↓
lee DATABASE_URL

alembic.ini
↓
configura migraciones
```

```
compose.yml
  ↓
levanta servicios

.env.example
  ↓
documenta variables

app.js
  ↓
apunta el frontend a http://localhost:8000
```

29 Errores habituales

Backend no responde

Síntoma:

```
curl: failed to connect to localhost port 8000
```

Posibles causas:

- Docker no está levantado.
- Uvicorn no está ejecutándose.
- Puerto 8000 incorrecto.
- Contenedor api fallando.

Error de base de datos

Síntoma:

```
/health no devuelve db ok
```

Posibles causas:

- PostgreSQL no está levantado.
- DATABASE_URL incorrecta.
- Host db usado fuera de Docker.
- Usuario o contraseña incorrectos.

CORS error en navegador

Síntoma:

```
CORS error
```

Posibles causas:

- Frontend no servido desde localhost:5173.
- Backend no permite ese origen.
- Se abrió el HTML directamente con file:///.

Alertas no aparecen

Posibles causas:

- No se han creado reglas.
- Los eventos no coinciden con las reglas.
- La alerta ya existe y el anti-duplicado la bloquea.

- La regla está disabled.
- Se está filtrando por status o group_key.

Docker copia archivos innecesarios

Si no hay `.dockerignore`, Docker podría copiar:

```
venv/  
__pycache__/  
.pytest_cache/  
*.pyc
```

Esto no impide necesariamente la ejecución, pero hace la imagen menos limpia.

Para la entrega ZIP ya se excluyeron estos elementos.

3.0 Relación con la entrega del proyecto

Para que el evaluador pueda ejecutar el proyecto, la entrega debe incluir:

```
backend/  
frontend/  
docker/  
.env.example  
README.md
```

Y excluir:

```
.env  
docker/.env  
.git/  
venv/  
__pycache__/  
.pytest_cache/  
*.pyc
```

La ejecución recomendada debería estar documentada en el README.

La opción más clara para entrega es:

```
docker compose -f docker/compose.yml up --build
```

Y para frontend:

```
cd frontend  
python3 -m http.server 5173
```

3.1 Resumen técnico

El laboratorio puede ejecutarse localmente o mediante Docker.

La opción Docker es la más reproducible porque levanta backend y PostgreSQL de forma coordinada.

El backend se sirve con Uvicorn en el puerto 8000.

El frontend se sirve en el puerto 5173 para coincidir con la configuración CORS.

La base de datos se configura mediante `DATABASE_URL`.

El flujo mínimo de validación es:

```
/health  
↓
```

```
crear regla
  ↓
ingestar evento
  ↓
consultar alerta
  ↓
actualizar estado
  ↓
consultar métricas
```

Con esta nota queda cerrado el bloque:

```
01_Entorno-de-desarrollo
└─ 02_Analisis-tecnico-del-entorno
   └─ 01_estructura-general-del-proyecto
   └─ 02_entorno-python-backend
   └─ 03_dependencias-y-requirements
   └─ 04_variables-de-entorno
   └─ 05_ejecucion-local-y-docker
```